Master Econométrie et Statistique Appliquée
2025–2026

Projet d'étude Master 2 ESA

FRAUD DETECTION

Rédigé par

LACROIX Ewan
ABOUBAKAR Ali Ibrahim
DIALLO Ousmane Djounnou

ESA MASTER ESA
Econométrie et Statistique Appliquée
Université d'Orléans

ORLÉANS SCHOOL OF ECONOMICS
OSE

Contents

1 Introduction

Nowadays, the fight against fraud is a major issue due to its increasing financial and social consequences. This problem is becoming more serious as the number of victims of scams and payment fraud continues to grow. In France, police and national gendarmerie services recorded 411,700 victims in 2023 and 417,300 in 2024, representing an increase of 1.4%. Although this growth has slowed down, it remains persistent[1]. This trend is partly explained by the diversification of fraud techniques, which has been reinforced by the recent development of technologies such as artificial intelligence.

Among the different types of fraud, credit card fraud is currently one of the most common, especially with the growth of online transactions since the Covid-19 health crisis. Credit card fraud refers to the unauthorized use of a payment card, or similar payment methods, in order to illegally obtain money or goods. In France, this type of fraud accounted for total declared losses of 519 million euros in 2024[2], highlighting the significant economic impact of this phenomenon.

One of the main challenges in fraud detection is that fraudulent transactions are both rare and constantly evolving. Fraudulent transactions represent only a very small proportion of all transactions, leading to a strong class imbalance in the datasets. In this context, traditional machine learning approaches tend to focus on the majority class of legitimate transactions, often at the expense of detecting fraudulent but critical events. In addition, fraud patterns change rapidly in order to bypass existing security systems, which requires the use of models that can continuously adapt. This double challenge limits the effectiveness of standard machine learning methods when no specific data rebalancing strategy or adaptive approach is applied.

In this report, we first perform a descriptive analysis of the data to identify the main characteristics of fraudulent transactions. We then apply unsupervised learning techniques to detect suspicious or anomalous transactions.In the second part, we evaluate the performance of several supervised learning methods. A third part is dedicated to the introduction of different resampling techniques, followed by a new implementation of the supervised models. Finally, we compare and analyze all the results in order to assess the impact of these different approaches on fraud detection.

[1] French Ministry of the Interior, *"Insecurity and Crime in 2024", 9^{th} statistical report.*
[2] Payment Methods Security Observatory, *Annual Report*, 2024.

2 Descriptive Analysis

The dataset contains **284,807 transactions** made by European cardholders over a period of 48 hours in September 2013. This structured dataset is complete and contains no missing values.

Most of the variables are obtained from a PCA transformation. Only two original variables are kept:

Time : time elapsed in seconds since the first transaction in the dataset.

Amount : transaction amount in euros.

The target variable **Class** is a binary indicator, where 0 corresponds to normal transactions and 1 to fraudulent transactions.

However, the data distribution reveals a **strong class imbalance**. With only 492 fraud cases, the minority class represents only 0.172% of the total number of transactions. This imbalance makes classical metrics such as accuracy misleading and requires the use of specific evaluation metrics and resampling techniques to avoid biased results.

2.1 Analysis of raw variables (Amount and Time)

The analysis of these two variables highlights behavioral differences between legitimate and fraudulent transactions. The table below summarizes the main descriptive statistics:

Table 1: Descriptive statistics

Variable	Mean	Median	Min	Max
Amount (€)	88.35	22.00	0.00	25,691.16
Time (hours)	26.34	23.53	0.00	48.00

The distribution of transaction amounts shows a large gap between the mean (88.35 €) and the median (22.00 €). This indicates that most transactions involve small amounts, while a few very large transactions, some exceeding 25,000 €, increase the average.

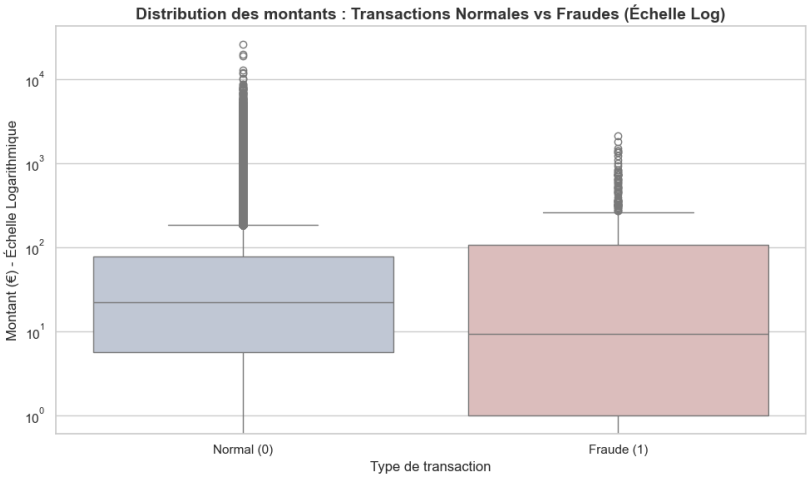


Figure 1: Amount distribution: Normal transactions vs Fraud (Log scale)

Normal vs Fraud comparison Fraudulent transactions exhibit a highly skewed distribution: the mean amount (122 €) is much higher than the median (9 €). This indicates that most frauds involve small amounts, with only a few large values.

In contrast, normal transactions are more evenly distributed around their median (22 €), although they also include some large amounts. As a result, the transaction amount alone is not sufficient to clearly distinguish fraudulent transactions from legitimate ones.

The `Time` variable allows us to observe transaction activity over two full days. For analysis purposes, it is more relevant to transform this variable to represent the actual hour of the transaction within a 24-hour cycle.

This transformation shows that legitimate transactions follow a typical daily pattern: low activity at night and higher activity during the day. The fraud profile, however, is quite different.

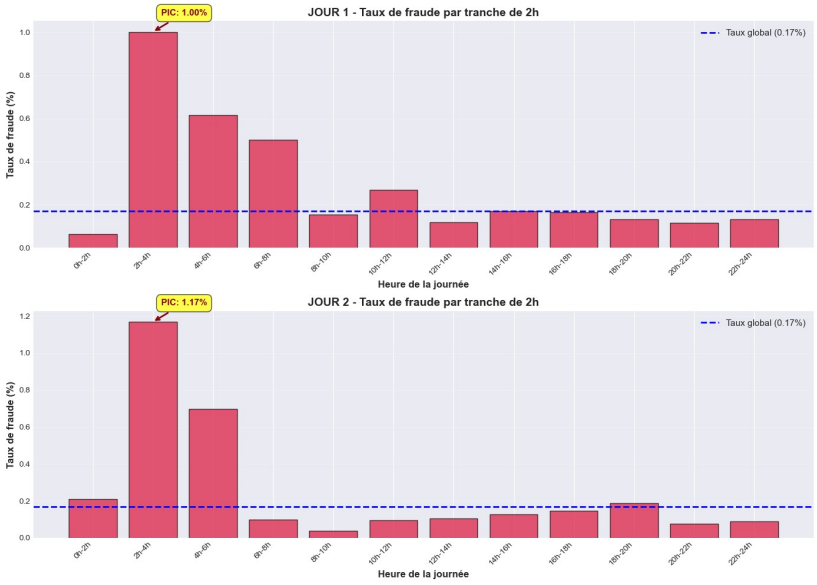


Figure 2: Fraud rate by hourly time slot over two days

As shown above, the fraud rate reaches a critical peak between 2 a.m. and 4 a.m. It rises to 1.00% during the first night and 1.17% during the second night, which is far above the global average (0.17%).

This pattern is repeated identically from one day to the next, highlighting a strong time-of-day effect: fraud attempts seem to target periods when cardholders are less vigilant (during sleep), delaying card blocking actions.

2.2 Selection of discriminative variables (Kruskal-Wallis test)

To assess the ability of the variables to distinguish fraudulent transactions from normal ones, Kruskal-Wallis tests were conducted. This non-parametric test compares the distributions of explanatory variables between the two groups.

The results show that some variables exhibit statistically significant differences between fraudulent and normal transactions (p-value < 0.05), while others do not show significant differences.

Table 2: Results of the Kruskal-Wallis test for selected variables

Variable	K-W statistic	p-value
V14	1189.04	< 0.0001
V4	1132.02	< 0.0001
V12	1125.74	< 0.0001
...	...	...
V15	2.30	0.1295
V22	1.24	0.2664

2.3 Multicollinearity analysis (VIF)

To analyze the relationships between variables, we use Spearman's rank correlation coefficient, which is more appropriate than Pearson's correlation when data do not follow a normal distribution.

The correlation analysis highlights a relationship between the `Amount` variable and the component **V2**, with a Spearman coefficient of $r_s = 0.50$. This suggests partial redundancy of information.

To quantify this effect, we compute the Variance Inflation Factor (VIF):

$$VIF_j = \frac{1}{1 - R_j^2} \tag{0}$$

The results confirm this observation:

- Variables **V1 to V28** have low VIF values (below the usual threshold of 5), confirming the independence of the principal components.
- The `Amount` variable, however, shows a high VIF (> 10), indicating strong correlation with V2.

Despite this high VIF, we retain the `Amount` variable due to its discriminative power. Moreover, ensemble models (such as Random Forest or XGBoost) handle correlated variables well, and **regularization** techniques can be applied if linear models are used.

2.4 Detection and Handling of Outliers

In this section we check for extreme values that might affect the model's performance, focusing especially on the `Amount` variable, which will be treated separately.

Univariate approach (IQR): Boxplot analysis shows that `Amount` has 11.19% extreme values, confirming high variability in transaction amounts. Principal components from PCA generally have low to moderate outlier rates.

Multivariate approaches: We compared several multivariate outlier detection methods, including Isolation Forest, Local Outlier Factor (LOF), One-Class SVM, Elliptic Envelope, and DBSCAN. These algorithms detect between 10% and 15% of outliers in the dataset, showing the presence of unusual behaviors in the data.

Treatment decision: Since the detected outlier rate is much higher than the usual 2% threshold for removal, we decided to keep them. We discretize `Amount` and `Time` using a decision tree (CART). This should reduce the impact of extreme values while keeping the variables useful for modeling.

3 Unsupervised Learning Methods

Fraud detection is generally based on labeled datasets, where each transaction is classified as fraudulent or legitimate. However, in real-world settings, these labels are not always reliable. In particular, there is a risk of double misspecification, which affects both the data and the models used for learning.

On the one hand, data misspecification occurs because some fraudulent transactions may be incorrectly labeled as non-fraudulent. This is mainly due to the adaptive nature of fraud: fraudsters actively try to hide their behavior by mimicking the characteristics of legitimate transactions in order to bypass detection systems. As a result, some frauds remain undetected during the labeling process, introducing noise into the labels. This type of error is especially problematic because false negatives (frauds labeled as non-fraud) are often much more frequent than the opposite case.

On the other hand, model misspecification comes from the assumption that the provided labels are correct and that the class distributions are well separated in the feature space. When fraudulent transactions are deliberately designed to look like normal transactions, this assumption is violated. In this situation, a supervised model may learn to treat certain fraudulent patterns as normal behavior, which reduces its ability to detect sophisticated or emerging frauds. This issue can also lead to a degradation of out-of-sample performance, especially when fraud strategies change over time.

Given this double challenge, unsupervised learning methods appear as a relevant alternative, or at least as a complement to supervised approaches. These methods do not rely on class labels during the training phase, which makes them naturally less sensitive to labeling errors. Instead of learning an explicit separation between fraud and non-fraud, they aim to model the underlying structure of the data and to identify atypical observations that may correspond to fraudulent behavior.

In this section, we apply two unsupervised algorithms Isolation Forest and HDBSCAN to identify suspicious or anomalous transactions and to evaluate their ability to detect fraud.

As shown in the following table, about 60% of the transactions identified as anomalous by Isolation Forest are actually fraudulent, while nearly 93% of the observations classified as noise by HDBSCAN correspond to fraudulent transactions.

4 Supervised Learning Methods

Supervised learning refers to a set of statistical and algorithmic methods used to model the relationship between a target variable and a set of explanatory variables, based on labeled observations. In fraud detection, the target variable is typically binary, indicating whether a transaction is fraudulent ($y = 1$) or legitimate ($y = 0$).

Let $\{(x_i, y_i)\}_{i=1}^n$ denote a training sample, where $x_i \in R^p$ is the feature vector of transaction i and $y_i \in \{0, 1\}$. The objective is to learn a decision function $f(x)$ that produces either a probability $\hat{p}(x) = P(y = 1 \mid x)$, or a score that is converted into a decision using a threshold τ, for example $1\{\hat{p}(x) \geq \tau\}$.

In this section, five supervised models are implemented and compared: logistic regression, Random Forest, AdaBoost, XGBoost, and a neural network.

4.1 Model presentation

4.1.1 Logistic regression

Logistic regression is a parametric probabilistic model widely used in econometrics for binary classification. It assumes that the conditional probability of fraud is given by

$$P(y = 1 \mid x) = \sigma(x^\top \beta) = \frac{1}{1 + \exp(-x^\top \beta)},$$

where $\beta \in R^p$ is a vector of parameters and $\sigma(\cdot)$ is the logistic function.

The parameters are estimated by maximizing the log-likelihood (equivalently, by minimizing the logistic loss). For a sample $\{(x_i, y_i)\}$, the negative log-likelihood is

$$\mathcal{L}(\beta) = -\sum_{i=1}^{n} \Big[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \Big], \quad \text{where } \hat{p}_i = \sigma(x_i^\top \beta).$$

Logistic regression offers direct interpretability. However, its main limitations in fraud detection come from its linear structure in the feature space and its limited ability to capture complex interactions without feature engineering (interaction terms, non-linear transformations). Despite this, it remains a strong baseline model, especially for calibration and benchmarking.

4.1.2 Random Forest

A Random Forest is an ensemble of decision trees trained using a bagging strategy. The procedure is as follows:

- Draw B bootstrap samples from the training dataset.
- Train a decision tree on each bootstrap sample.
- At each node of a tree, randomly select a subset of variables (of size m) and consider only these variables to find the best split.

The final prediction aggregates the individual trees, either by majority vote or by averaging predicted probabilities:

$$\hat{p}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{p}_b(x).$$

Random Forests naturally capture non-linear relationships and interactions, and reduce variance through aggregation. They perform well without explicit parametric assumptions. However, the resulting models can be large, and their global interpretability is lower than that of logistic regression.

4.1.3 AdaBoost

AdaBoost is a boosting method that sequentially combines weak classifiers, often shallow decision trees. The main idea is to give more weight to misclassified observations in order to focus learning on difficult cases.

Formally, a distribution of weights $\{w_i^{(t)}\}$ is maintained over the observations. At each iteration t, a weak classifier $h_t(x)$ is trained to minimize the weighted error, and it is assigned a weight α_t that increases with its performance. The final prediction is a weighted sum:

$$F(x) = \sum_{t=1}^{T} \alpha_t h_t(x), \quad \hat{y} = 1\{F(x) \geq 0\},$$

and a probabilistic output can be obtained via a transformation, depending on the implementation.

AdaBoost can be effective when simple rules, combined sequentially, are sufficient to separate fraud from non-fraud. However, it can be sensitive to noise and outliers, since misclassified observations receive increasing weight over iterations, which may harm generalization if these points correspond to labeling errors rather than informative fraud cases.

4.1.4 XGBoost

XGBoost belongs to the family of gradient boosting tree methods. It builds an additive model of the form

$$\hat{y}(x) = \sum_{t=1}^{T} f_t(x), \quad f_t \in \mathcal{F},$$

where each f_t is a decision tree. At iteration t, a new tree is added to correct the residual errors of the current model by minimizing an objective function composed of a prediction loss and a regularization term:

$$\mathcal{J} = \sum_{i=1}^{n} \ell(y_i, \hat{y}_i) + \sum_{t=1}^{T} \Omega(f_t).$$

The regularization term $\Omega(\cdot)$ penalizes model complexity, such as the number of leaves and leaf weights, which helps prevent overfitting.

XGBoost also includes learning rate control, subsampling techniques, and efficient optimization for large datasets.

In highly non-linear and imbalanced problems such as fraud detection, XGBoost is widely used due to its strong bias–variance trade-off and its ability to model complex interactions. Its main limitations are sensitivity to hyperparameter tuning and the need for careful validation.

4.1.5 Neural network

A neural network of the multilayer perceptron (MLP) type approximates a non-linear function through a composition of layers. For a network with one hidden layer, we can write:

$$h(x) = \phi(W_1 x + b_1), \quad \hat{p}(x) = \sigma(W_2 h(x) + b_2),$$

where W_1, W_2 are weight matrices, b_1, b_2 are bias vectors, ϕ is an activation function, and σ is the sigmoid function used to produce a probability.

Training is performed by minimizing a loss function (usually cross-entropy) using stochastic gradient descent and backpropagation. Regularization techniques are required to prevent overfitting, such as L_2 penalization, early stopping, and possibly dropout.

Neural networks are attractive when the relationship between features and fraud is complex, involving many non-linear effects and interactions. However, for classical tabular data, they require careful tuning (architecture, learning rate, normalization, regularization) and offer lower interpretability compared to simpler models.

4.2 Performance measures and comparison criteria

After hyperparameter optimization for each model and evaluation on the test sample, performance metrics are computed in order to compare the models. We consider the following measures:

Recall (sensitivity, true positive rate). Recall measures the ability of the model to detect frauds that are actually present:

$$\text{Recall} = \frac{TP}{TP + FN}.$$

A high recall limits missed frauds, which is generally critical when the cost of false negatives is high.

Precision (positive predictive value). Precision measures the reliability of the alerts issued by the model:

$$\text{Precision} = \frac{TP}{TP + FP}.$$

A high precision implies few false alerts among the detected cases, which is important when human investigation is costly or when automatic actions (such as blocking transactions) are triggered.

Accuracy (overall classification rate). Accuracy is the overall proportion of correct predictions:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

In a highly imbalanced context (rare fraud), accuracy can be misleading, as always predicting the majority class may already yield a high accuracy.

G-Mean (geometric mean). The G-Mean seeks a balance between performance on the minority class (fraud) and the majority class (non-fraud). First, specificity is defined as

$$\text{Specificity} = \frac{TN}{TN + FP},$$

and then

$$\text{G-Mean} = \sqrt{\text{Recall} \times \text{Specificity}}.$$

This metric penalizes extreme solutions, such as models that detect almost everything but generate too many false positives, or the opposite.

AUC-ROC. The ROC curve plots TPR = Recall against FPR = $\frac{FP}{FP+TN}$ for all possible thresholds. The area under this curve (AUC-ROC) measures the ranking ability of the model and can be interpreted as the probability that a randomly chosen positive example receives a higher score than a randomly chosen negative example:

$$\text{AUC-ROC} = P(s(X^+) > s(X^-)).$$

AUC-ROC is useful for comparing models in terms of overall discriminative power, but it can remain high even when operational precision is low in extremely imbalanced datasets.

AUC-PR. The Precision–Recall curve plots precision as a function of recall for all thresholds. The area under this curve (AUC-PR) is often more relevant when the positive class is rare, since precision is directly affected by the proportion of false positives among alerts. A useful reference is the baseline level equal to the prevalence $\pi = P(y = 1)$: a random classifier has an expected precision close to π, regardless of recall.

Cross-Entropy (log loss). Cross-entropy evaluates probabilistic quality via likelihood, strongly penalizing confident errors. For predicted probabilities $\hat{p}_i \in (0, 1)$:

$$\text{CE} = -\frac{1}{n} \sum_{i=1}^{n} \left[y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i) \right].$$

Lower cross-entropy indicates better probabilistic predictions under the logarithmic loss.

Brier score. The Brier score measures the mean squared error of predicted probabilities:

$$\text{Brier} = \frac{1}{n}\sum_{i=1}^{n}(\hat{p}_i - y_i)^2.$$

A low Brier score indicates probabilities close to observed frequencies and thus better calibration, although calibration can be studied more finely using calibration curves.

4.3 Results without resampling

4.3.1 Training protocol

The initial dataset, composed of 284,807 observations, is split into two disjoint subsets using a stratified split based on the fraud variable (positive class), in order to preserve the same proportion of fraudulent transactions in each subset. More precisely, 70% of the data are allocated to training and 30% are reserved for a test set, which is kept exclusively for final out-of-sample evaluation.

Hyperparameter selection is performed exclusively within the training portion (70%), which is itself subdivided into two subsets, again using stratification by fraud: 75% for model fitting and 25% for a validation (or calibration) set used to compare hyperparameter configurations under an internal hold-out scheme. The comparison of the five models is then conducted based solely on their performance measured on the test set (30%), while the validation set is used only for hyperparameter optimization.

4.3.2 Results

The following table summarizes the results of the different models evaluated on the test sample of the original dataset:

Table 3: Comparative table of fraud detection models.

Model	Recall	Precision	AUC ROC	AUC PR	Accuracy	G Mean	Cross Entropy	Brier Score	F1 Score
LogisticRegression	0.770 270 3	0.826 087 0	0.975 878 2	0.753 066 5	0.999 321 2	0.877 526 9	0.102 215 5	0.021 304 0	0.797 202 8
RandomForest	0.790 540 5	0.921 259 8	0.959 513 7	0.836 718 4	0.999 520 1	0.889 071 3	0.006 592 4	0.000 471 2	0.850 909 1
NeuralNet	0.750 000 0	0.932 773 1	0.985 042 9	0.832 121 7	0.999 473 3	0.865 984 8	0.002 866 4	0.000 490 6	0.831 460 7
AdaBoost	0.797 297 3	0.797 297 3	0.970 957 8	0.713 417 4	0.999 297 8	0.892 758 0	0.169 694 9	0.028 006 4	0.797 297 3
XGBoost	0.790 540 5	0.921 259 8	0.977 409 7	0.812 555 5	0.999 520 1	0.889 071 3	0.007 439 5	0.001 169 6	0.850 909 1

Using the original dataset without resampling, our best-performing model is the Random Forest. We prioritize the area under the Precision–Recall curve and, consequently, the F1-score. Although other models achieve higher AUC-ROC values, in fraud detection the key metrics are recall and precision. Given the relatively low cost of fraud in this dataset, it is also necessary to maintain a satisfactory precision in order to avoid excessive investigation costs.

The Random Forest satisfies these requirements, while XGBoost follows closely, with a better Brier score but slightly lower AUC-PR and higher AUC-ROC. Recall that for all these criteria, the selected threshold is the one that maximizes the F1-score.

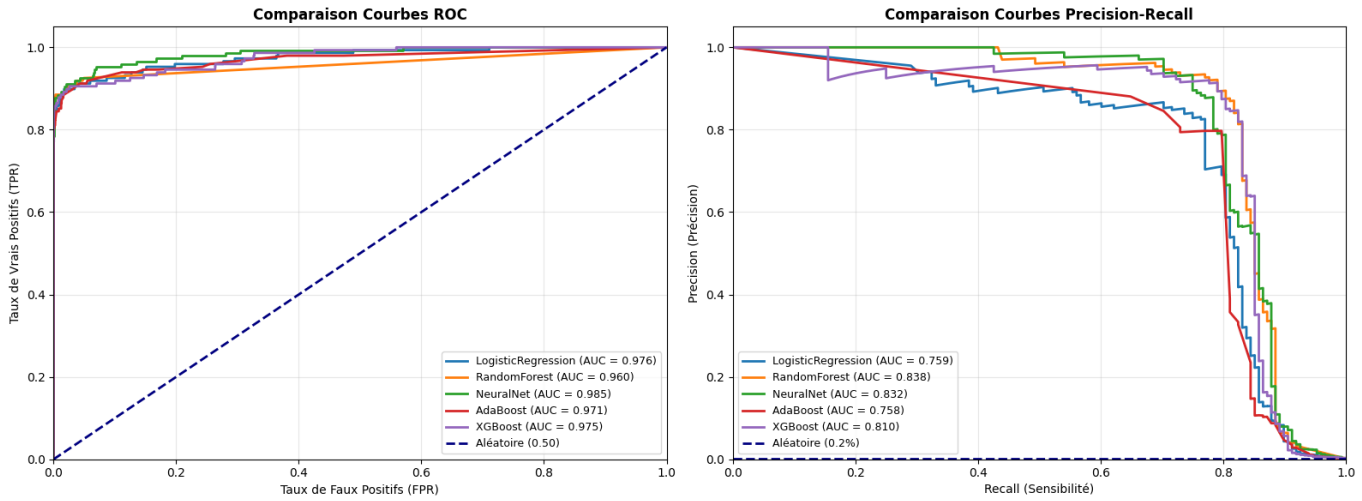


Figure 3: Comparison of ROC and Precision–Recall curves

Overall, the models exhibit very similar performance, and the final choice depends on the trade-off discussed above. On the original dataset, it would therefore be reasonable to retain either XGBoost or Random Forest.

4.4 Handelling class imbalance

Class imbalance is a major issue in fraud detection. In our dataset, fraudulent transactions represent only 1.7% of the observations. This strong asymmetry can bias the learning process of classification models.

In order to address this imbalance, we applied several resampling methods using three target ratios: 5%/95%, 10%/90%, and 20%/80% (minority class / majority class). We considered three main families of resampling methods: over-sampling, under-sampling, and hybrid methods. These approaches aim respectively to increase the number of minority class observations, reduce the number of majority class observations, or combine both strategies.

4.4.1 Resampling Methods

Over-sampling methods consist in increasing the number of minority class observations in order to rebalance the class distribution. In this category, we used the following techniques:

- **Random Over-Sampling (ROS)**: This method randomly duplicates observations from the minority class until the desired ratio is reached. It is simple to implement, but it may lead to overfitting due to the exact repetition of the same observations [**?**].
- **Synthetic Minority Over-sampling Technique (SMOTE)**: The SMOTE method [Chawla et al., 2002] generates new synthetic observations based on existing minority class samples. For each observation, one of its k nearest neighbors is selected and a new sample is created by linear interpolation. This approach allows for better coverage of the minority class feature space [Tremblay et al., 2022].
- **Adaptive Synthetic Sampling (ADASYN)**: ADASYN [He et al., 2008] is an extension of SMOTE that generates more synthetic observations in regions where the minority class is poorly represented. The over-sampling process is therefore adapted to the local difficulty of the problem, improving the modeling of complex regions.
- **Borderline-SMOTE**: This variant of SMOTE focuses on minority class observations located near the decision boundary. These observations, which are mostly surrounded by majority class samples, are considered particularly informative for improving the discriminative ability of the model [**?**].
- **SVM-SMOTE**: SVM-SMOTE uses a decision boundary estimated with a SVM classifier to identify minority class observations close to the margin. Synthetic samples are then generated around these points in order to reinforce the separation between the two classes [**?**].

Under-sampling methods aim to reduce the number of majority class observations in order to rebalance the class distribution. In our study, we used the following techniques:

- **Random Under-Sampling (RUS)**: This method randomly removes observations from the majority class until the target ratio is reached. While it reduces computational cost, it may lead to information loss if relevant observations are removed.
- **NearMiss**: NearMiss selects majority class observations based on their distance to minority class samples. We use version 1, which keeps the majority observations that are closest to the minority class. This strategy helps preserve the most informative samples near the decision boundary [**?**].

Hybrid methods combine over-sampling and under-sampling techniques in order to benefit from the advantages of both approaches. In this context, we used the following methods:

- **SMOTE-ENN**: This method combines SMOTE over-sampling with data cleaning using Edited Nearest Neighbors (ENN). After generating synthetic samples, observations whose class differs from the majority of their k nearest neighbors are removed. This approach leads to a more robust decision boundary [Batista et al., 2004].
- **Condensed Nearest Neighbor (CNN)**: The CNN method [Gowda and Krishna, 1979] aims to reduce the size of the training set while preserving discriminative information. The algorithm initializes the set with all minority class observations and then adds only the majority class observations necessary for correct classification. The result is a more compact and representative subset.

For each resampling method, we generated three training datasets corresponding to the ratios 5%/95%, 10%/90%, and 20%/80%, except for the CNN method, which does not rely on a predefined ratio. In total, 25 training datasets were constructed (8 methods $\times$ 3 ratios + CNN). The test set is kept unchanged and imbalanced in order to evaluate model performance under conditions close to a real-world setting.

4.4.2 Results

All results are reported in tables in the appendix, allowing us to compare all our models across all datasets, for a total of 130 models. We observe that, regardless of the dataset, penalized logistic regression always performs worse than the other models. Random Forest and XGBoost rank first, which is expected given their model structure.

We also note that resampling improved the results, especially for XGBoost and for all models using SMOTE methods. In particular, the standard SMOTE method with a 10/90 sampling ratio provides the best performance with the XGBoost model. The results of models trained on this dataset are presented in the following table:

Table 4: Comparison table of models for the SMOTE 10/90 configuration.

Model	Recall	Precision	AUC ROC	AUC PR	Accuracy	G Mean	Cross Entropy	Brier Score	F1 Score
AdaBoost	0.756 756 8	0.794 326 2	0.975 078 5	0.722 831 2	0.999 239 3	0.869 769 8	0.191 329 6	0.035 054 8	0.775 086 5
LogisticRegression	0.777 027 0	0.839 416 1	0.974 977 0	0.749 008 0	0.999 356 3	0.881 377 7	0.049 747 2	0.008 464 8	0.807 017 5
NeuralNet	0.722 973 0	0.930 434 8	0.950 320 1	0.796 111 7	0.999 426 5	0.850 238 3	0.009 179 8	0.000 613 7	0.813 688 2
RandomForest	0.790 540 5	0.886 363 6	0.977 747 0	0.841 736 8	0.999 461 6	0.889 045 3	0.009 271 7	0.001 046 3	0.835 714 3
XGBoost	0.810 810 8	0.937 500 0	0.967 424 4	0.852 445 2	0.999 578 7	0.900 408 1	0.003 811 6	0.000 499 0	0.869 565 2

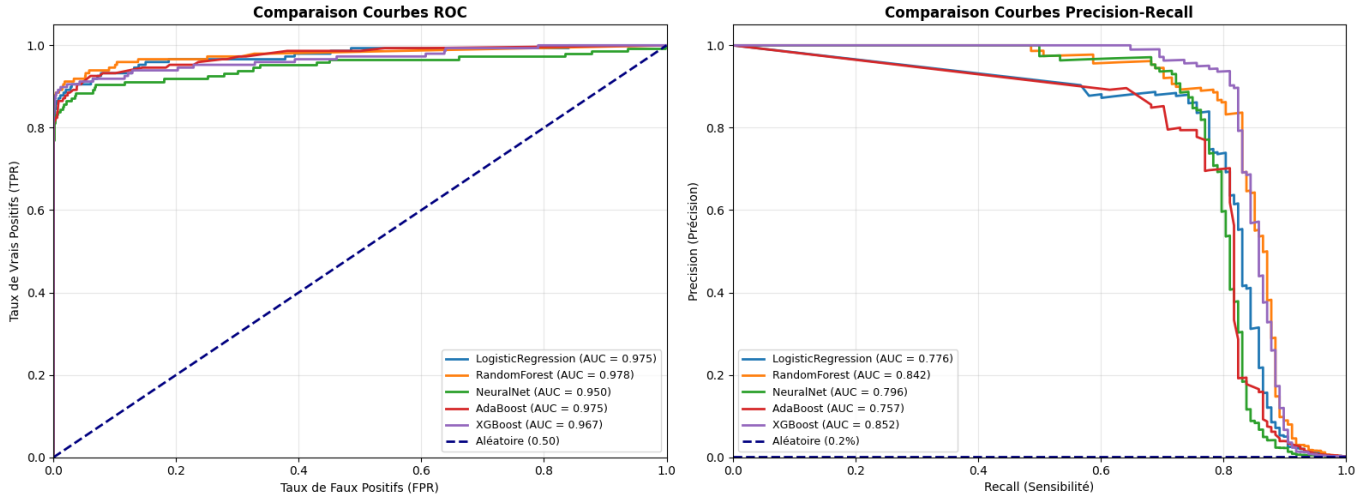


Figure 4: Comparison of AUC ROC and PR curves | SMOTE 10/90

XGBoost clearly achieves very strong results compared to logistic regression and AdaBoost.

Therefore, we recommend using the XGBoost model trained with SMOTE 10/90 for fraud detection. It achieves a recall of 81% and a precision of 93.75%. Given that the average fraud amount is relatively low (around 120 euros), a trade-off between investigation cost and fraud cost must be considered. For this reason, the F1-score is an important metric, which is clearly reflected in the confusion matrix provided in the appendix.

5 Cost-Sensitive Learning Approach

The classification methods used so far rely on an implicit assumption: all classification errors have the same cost. In other words, a false positive (a legitimate transaction classified as fraudulent) and a false negative (a fraudulent transaction classified as legitimate) are treated as equivalent during model optimization. This assumption is clearly reflected in the loss function of standard logistic regression, which assigns the same weight to all observations, regardless of the actual financial consequences of a classification error.

In the context of fraud detection, this assumption may be unrealistic. Indeed, the costs associated with different types of errors are not the same. On the one hand, a false positive mainly generates administrative and investigation costs: the monitoring team must manually review the flagged transaction, potentially contact the customer, and devote time and resources to verification. On the other hand, a false negative results in a direct financial loss corresponding to the amount of the undetected fraudulent transaction.

Moreover, the benefits of correct classification also differ. A true negative (a legitimate transaction correctly classified) generates no additional cost. In contrast, a true positive does incur an investigation cost, but it prevents a potentially significant financial loss. This cost structure fully justifies the use of a cost-sensitive approach.

To formalize this problem, we define a cost matrix $C_i(\hat{y}|y)$, which represents the cost associated with predicting class $\hat{y}$ when the true class is y, for transaction i. This matrix is defined as follows:

	Predicted fraud ($\hat{y} = 1$)	**Predicted legitimate** ($\hat{y} = 0$)
True fraud ($y = 1$)	$C_i(1\|1) = c_f$ (True positive)	$C_i(0\|1) = A_i$ (False negative)
True legitimate ($y = 0$)	$C_i(1\|0) = c_f$ (False positive)	$C_i(0\|0) = 0$ (True negative)

Table 5: Cost matrix

where:

- c_f represents the fixed investigation cost applied to any transaction flagged as suspicious (whether it is truly fraudulent or not).
- A_i denotes the variable cost of a false negative for transaction i, corresponding to the amount of the undetected fraudulent transaction.
- The cost of a true negative is zero ($C_i(0|0) = 0$), since a legitimate transaction correctly classified does not generate any additional cost.

Given this cost matrix, we define the total cost associated with a classification model with score function $\pi(\cdot)$ on a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$. For transaction i, the expected cost is expressed as a function of the predicted probability $\pi_i = \pi(\mathbf{x}_i)$ and the associated costs:

$$E[\text{Cost}_i] = y_i[\hat{y}_i C_i(1|1) + (1 - \hat{y}_i)C_i(0|1)] + (1 - y_i)[\hat{y}_i C_i(1|0) + (1 - \hat{y}_i)C_i(0|0)]$$

By substituting the values from the cost matrix and using the conditional expectation $E[\hat{y}_i|\mathbf{x}_i] = \pi_i$, we obtain:

$$E[\text{Cost}_i] = y_i[\pi_i c_f + (1 - \pi_i)A_i] + (1 - y_i)[\pi_i c_f + 0]$$

Expanding this expression yields:

$$E[\text{Cost}_i] = y_i \pi_i c_f + y_i(1 - \pi_i)A_i + (1 - y_i)\pi_i c_f$$

$$E[\text{Cost}_i] = y_i A_i - y_i \pi_i A_i + \pi_i c_f[y_i + (1 - y_i)]$$

$$E[\text{Cost}_i] = y_i A_i(1 - \pi_i) + \pi_i c_f$$

The average expected cost (AEC) over the dataset is then defined as:

$$\boxed{AEC(\boldsymbol{\beta}) = \frac{1}{N}\sum_{i=1}^{N}[y_i(1 - \pi_i)A_i + \pi_i c_f]}$$

where $\pi_i = \pi(\mathbf{x}_i) = \frac{e^{\mathbf{x}_i^T \boldsymbol{\beta}}}{1+e^{\mathbf{x}_i^T \boldsymbol{\beta}}}$ is the probability predicted by the logistic regression model.

The objective of cost-sensitive logistic regression (CSLogit) is to find the parameters $\boldsymbol{\beta}$ that minimize the average expected cost:

$$\boldsymbol{\beta}^* = \arg\min_{\boldsymbol{\beta}} AEC(\boldsymbol{\beta})$$

To solve this optimization problem, we compute the gradient of the AEC with respect to the parameters $\boldsymbol{\beta}$. Differentiating the AEC yields:

$$\frac{\partial AEC}{\partial \boldsymbol{\beta}} = \frac{1}{N}\sum_{i=1}^{N}\left[-y_i A_i \frac{\partial \pi_i}{\partial \boldsymbol{\beta}} + c_f \frac{\partial \pi_i}{\partial \boldsymbol{\beta}}\right]$$

$$\frac{\partial AEC}{\partial \boldsymbol{\beta}} = \frac{1}{N}\sum_{i=1}^{N}(c_f - y_i A_i)\frac{\partial \pi_i}{\partial \boldsymbol{\beta}}$$

where

$$\frac{\partial \pi_i}{\partial \boldsymbol{\beta}} = \pi_i(1 - \pi_i)\mathbf{x}_i$$

Substituting this expression gives the gradient:

$$\boxed{\nabla_{\boldsymbol{\beta}} AEC = \frac{1}{N}\sum_{i=1}^{N}\pi_i(1 - \pi_i)(c_f - y_i A_i)\mathbf{x}_i}$$

However, using this approach leads to very low predicted fraud probabilities for high investigation costs and very high predicted probabilities for low investigation costs. To address this issue, we adopt an intermediate approach that not only minimizes the average fraud cost but also maximizes the likelihood.

The composite objective function is defined as:

$$\mathcal{L}(\boldsymbol{\beta}) = \alpha \cdot AEC(\boldsymbol{\beta}) - (1-\alpha) \cdot \ell(\boldsymbol{\beta})$$

where $\alpha \in [0,1]$ is a hyperparameter controlling cost sensitivity, and $\ell(\boldsymbol{\beta})$ denotes the standard logistic regression log-likelihood.

Table 6 below presents the results obtained by the model for different values of the fixed investigation cost c_f:

Table 6: Performance of the CSLogit model for different values of the fixed investigation cost c_f

c_f	Recall	Precision	AUC-ROC	AUC-PR	Accuracy	G-Mean	Cross-Entropy	Brier Score	F1-Score
5	0.601	0.832	0.950	0.672	0.999	0.775	0.0059	0.00089	0.698
15	0.595	0.889	0.948	0.665	0.999	0.771	0.0052	0.00080	0.713
25	0.723	0.856	0.941	0.697	0.999	0.850	0.0084	0.00070	0.784
35	0.716	0.876	0.938	0.691	0.999	0.846	0.0085	0.00071	0.788
45	0.716	0.869	0.935	0.679	0.999	0.846	0.0087	0.00071	0.785
55	0.716	0.855	0.932	0.670	0.999	0.846	0.0088	0.00070	0.779
65	0.547	0.827	0.913	0.576	0.999	0.740	0.0075	0.00099	0.659

We observe that for low investigation costs, the model prioritizes precision at the expense of recall: with $c_f = 5$, only 60% of frauds are detected, but with high precision (83–89%).

Although the performance of CSLogit remains modest compared to the best models obtained in previous stages (notably in terms of AUC-ROC and recall), it remains satisfactory and has the advantage of explicitly incorporating the cost structure into the optimization process.

Here are the total costs calculated on the test set using the investigation costs from training:

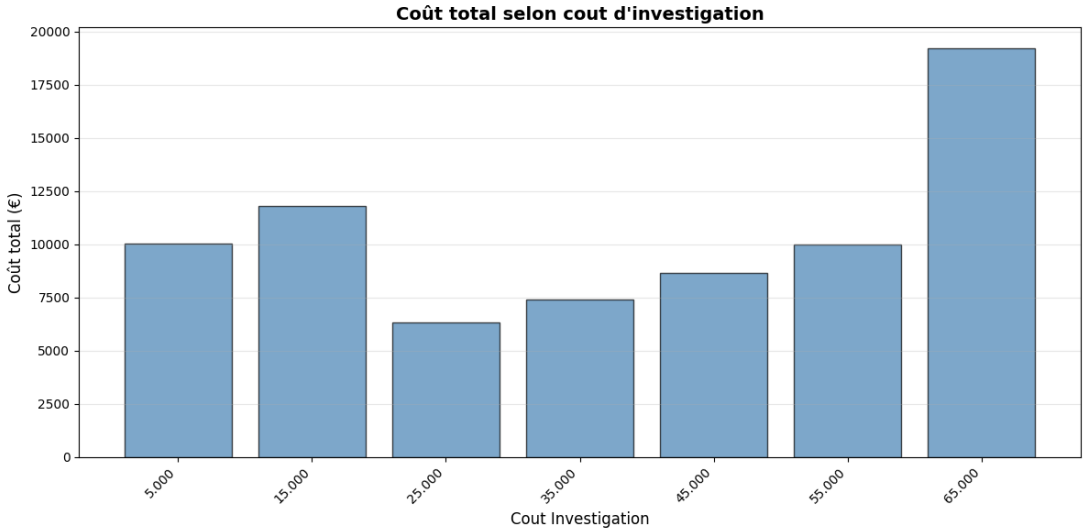


Figure 5: Fraud cost according to investigation cost

6 Conclusion

Fraud detection represents a major challenge due to its rare and evolving nature. In this project, we implemented and compared several supervised and unsupervised learning techniques, as well as various resampling methods to address class imbalance.

Unsupervised methods proved capable of detecting fraud to a certain extent. Isolation Forest identified 68% of fraudulent transactions among the detected anomalies, while HDBSCAN achieved even higher performance, with 84.15% of frauds classified as anomalies.

Supervised models, however, achieved significantly better results, particularly Random Forest and XGBoost. Due to their structure, these algorithms are able to model complex non-linear relationships and interactions between variables, unlike logistic regression, which remains limited by its linear nature. Resampling proved to be important for improving performance: by rebalancing the classes, these techniques allowed models to better learn the characteristics of the minority class (fraud) and thus significantly improve detection capability.

The results obtained could be further improved through a more exhaustive exploration of the hyperparameter space and finer model tuning. In addition, other architectures were not tested in this work and could potentially offer superior performance.

The final model choice results from a trade-off between investigation costs and fraud costs, while also accounting for potential regulatory detection requirements. In a context where detection obligations are paramount, it would be appropriate to increase recall at the expense of precision in order to capture as many fraudulent transactions as possible, even if this generates more false alerts. However, within the scope of our analysis, we favored a profitability-oriented approach aimed at optimizing the cost–benefit trade-off.

Considering all results, we conclude that the XGBoost model trained on the SMOTE 10/90 dataset represents the best compromise, with a recall of 81%, a precision of 93.75%, and an F1-score of 0.870. This choice could be further refined if a precise estimate of investigation costs were available, allowing this information to be explicitly integrated into the decision-making process. In its current form, this model offers the best balance between detection performance and alert reliability, given the information at hand.

References

[Banulescu-Radu, 2025] Banulescu-Radu, D. (2025). Fraud detection. Course notes.

[Batista et al., 2004] Batista, G. E., Prati, R. C., and Monard, M. C. (2004). A study of the behavior of several methods for balancing machine learning training data. *ACM SIGKDD Explorations Newsletter*, 6(1):20–29.

[Chawla et al., 2002] Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. (2002). Smote: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357.

[Gowda and Krishna, 1979] Gowda, K. C. and Krishna, G. (1979). The condensed nearest neighbor rule using the concept of mutual nearest neighborhood. *IEEE Transactions on Information Theory*, 25(4):488–490.

[He et al., 2008] He, H., Bai, Y., Garcia, E. A., and Li, S. (2008). Adasyn: Adaptive synthetic sampling approach for imbalanced learning. *IEEE International Joint Conference on Neural Networks*, pages 1322–1328.

[Tremblay et al., 2022] Tremblay, M. et al. (2022). Extensions de smote pour données mixtes. *Revue d'Intelligence Artificielle*.

[Yankol Schalck, 2025] Yankol Schalck, M. (2025). Auto insurance fraud detection: Leveraging cost sensitive and insensitive algorithms for comprehensive analysis. *Insurance: Mathematics and Economics*, 122:44–60.

7 Annexes

Table 7: Tableau comparatif des configurations de LogisticRegression pour la détection de fraude.

Configuration	Recall	Precision	AUC ROC	AUC PR	Accuracy	G Mean	Cross Entropy	Brier Score	F1 Score
Initiale	0.770 270 3	0.826 087 0	0.975 878 2	0.753 066 5	0.999 321 2	0.877 526 9	0.102 215 5	0.021 304 0	0.797 202 8
adasyn_05_95	0.777 027 0	0.839 416 1	0.976 162 4	0.735 771 7	0.999 356 2	0.881 378 3	0.053 741 6	0.009 072 1	0.807 017 5
adasyn_10_90	0.777 027 0	0.839 416 1	0.974 804 6	0.739 954 6	0.999 356 2	0.881 378 3	0.051 039 0	0.008 706 2	0.807 017 5
adasyn_20_80	0.756 756 8	0.868 217 0	0.974 743 6	0.750 667 4	0.999 379 7	0.869 830 8	0.052 340 7	0.009 072 0	0.808 664 3
borderlinesmote_05_95	0.797 297 3	0.761 290 3	0.958 320 6	0.747 364 5	0.999 215 6	0.892 720 7	0.024 340 7	0.003 477 3	0.778 878 0
borderlinesmote_10_90	0.783 783 8	0.811 188 8	0.950 621 4	0.747 717 4	0.999 309 5	0.885 176 0	0.017 840 7	0.002 830 6	0.797 251 4
borderlinesmote_20_80	0.777 027 0	0.809 859 2	0.947 757 2	0.744 361 4	0.999 298 2	0.881 352 4	0.016 500 7	0.002 640 4	0.793 103 4
cnn	0.750 000 0	0.685 185 2	0.970 366 2	0.708 220 3	0.998 970 4	0.865 766 5	0.112 971 4	0.021 002 7	0.716 129 0
nearmiss_05_95	0.432 432 4	0.118 738 3	0.962 753 6	0.084 658 2	0.993 458 1	0.655 761 9	0.422 960 7	0.067 864 0	0.186 317 0
nearmiss_10_90	0.445 945 9	0.150 342 5	0.961 382 9	0.105 684 2	0.994 675 4	0.666 330 4	0.335 570 8	0.067 947 3	0.224 872 1
nearmiss_20_80	0.398 648 6	0.046 862 8	0.945 222 6	0.033 569 5	0.984 913 9	0.626 928 7	0.832 110 3	0.108 863 1	0.083 866 4
randomoversampling_05_95	0.743 243 2	0.859 375 0	0.976 054 2	0.750 944 1	0.999 344 7	0.862 025 3	0.101 740 7	0.020 987 6	0.797 101 4
randomoversampling_10_90	0.736 486 5	0.865 079 4	0.976 663 6	0.744 912 8	0.999 344 7	0.858 102 5	0.102 183 6	0.021 162 7	0.795 620 4
randomoversampling_20_80	0.770 270 3	0.826 087 0	0.976 238 8	0.744 042 1	0.999 321 2	0.877 526 9	0.101 957 6	0.021 142 6	0.797 202 8
randomundersampling_05_95	0.682 432 4	0.677 852 3	0.969 526 9	0.674 007 7	0.998 887 7	0.825 862 1	0.152 017 2	0.028 006 5	0.680 134 7
randomundersampling_10_90	0.736 486 5	0.612 359 6	0.974 871 9	0.648 530 6	0.998 736 0	0.857 841 4	0.117 064 9	0.023 628 4	0.668 711 7
randomundersampling_20_80	0.729 729 7	0.548 223 4	0.970 642 9	0.595 492 5	0.998 490 2	0.853 796 3	0.118 200 1	0.022 155 4	0.626 087 0
smote_05_95	0.777 027 0	0.845 588 2	0.975 049 8	0.734 685 6	0.999 367 9	0.881 378 3	0.050 574 6	0.008 599 3	0.809 859 2
smote_10_90	0.777 027 0	0.839 416 1	0.974 976 5	0.749 007 9	0.999 356 2	0.881 378 3	0.049 746 9	0.008 464 9	0.807 017 5
smote_20_80	0.777 027 0	0.839 416 1	0.974 709 1	0.747 895 9	0.999 356 2	0.881 378 3	0.049 453 3	0.008 403 8	0.807 017 5
smoteenn_05_95	0.763 513 5	0.843 283 6	0.958 420 2	0.744 732 0	0.999 344 7	0.873 685 2	0.029 448 3	0.004 840 4	0.801 418 4
smoteenn_10_90	0.756 756 8	0.854 961 8	0.960 834 2	0.744 535 6	0.999 356 2	0.869 821 2	0.034 959 4	0.005 386 6	0.802 867 1
smoteenn_20_80	0.743 243 2	0.873 015 9	0.960 885 1	0.733 092 1	0.999 367 9	0.862 035 3	0.039 439 7	0.006 288 4	0.802 919 7
svmsmote_05_95	0.777 027 0	0.756 578 9	0.959 566 3	0.742 245 1	0.999 180 7	0.881 300 3	0.021 193 6	0.002 754 2	0.766 666 7
svmsmote_10_90	0.777 027 0	0.787 671 2	0.952 515 4	0.748 559 8	0.999 250 6	0.881 331 3	0.014 803 5	0.002 236 4	0.782 312 9
svmsmote_20_80	0.750 000 0	0.810 219 0	0.948 845 3	0.743 834 4	0.999 262 5	0.865 892 8	0.013 784 2	0.002 102 4	0.778 947 4

Table 8: Tableau comparatif des configurations de RandomForest pour la détection de fraude.

Configuration	Recall	Precision	AUC ROC	AUC PR	Accuracy	G Mean	Cross Entropy	Brier Score	F1 Score
Initiale	0.790 540 5	0.921 259 8	0.959 513 7	0.836 718 4	0.999 520 1	0.889 071 3	0.006 592 4	0.000 471 2	0.850 909 1
adasyn_05_95	0.831 081 1	0.860 139 9	0.966 870 7	0.822 519 3	0.999 473 3	0.911 530 1	0.008 836 4	0.000 705 8	0.845 360 8
adasyn_10_90	0.810 810 8	0.882 352 9	0.957 872 5	0.836 898 3	0.999 485 0	0.900 366 4	0.009 062 3	0.000 782 0	0.845 070 4
adasyn_20_80	0.797 297 3	0.893 939 4	0.981 167 0	0.850 709 7	0.999 485 0	0.892 841 9	0.007 729 4	0.000 891 9	0.842 857 1
borderlinesmote_05_95	0.777 027 0	0.905 511 8	0.976 683 6	0.822 086 4	0.999 473 3	0.881 429 0	0.004 615 9	0.000 570 7	0.836 363 6
borderlinesmote_10_90	0.797 297 3	0.887 218 0	0.979 857 9	0.820 012 1	0.999 473 3	0.892 837 2	0.004 391 6	0.000 585 1	0.839 858 4
borderlinesmote_20_80	0.783 783 8	0.920 634 9	0.956 512 2	0.824 606 4	0.999 508 4	0.885 263 7	0.007 035 3	0.000 506 3	0.846 715 3
cnn	0.783 783 8	0.865 671 6	0.979 629 4	0.752 424 6	0.999 414 8	0.885 222 5	0.036 302 4	0.004 308 8	0.822 695 0
nearmiss_05_95	0.797 297 3	0.825 174 8	0.973 286 8	0.752 726 3	0.999 356 2	0.892 784 4	0.042 173 2	0.007 798 9	0.810 996 6
nearmiss_10_90	0.763 513 5	0.869 230 8	0.972 169 4	0.742 364 9	0.999 391 4	0.873 705 7	0.108 537 6	0.033 093 1	0.812 949 6
nearmiss_20_80	0.797 297 3	0.861 313 9	0.961 118 3	0.720 531 8	0.999 426 5	0.892 815 8	0.213 818 2	0.072 132 4	0.828 070 2
randomoversampling_05_95	0.790 540 5	0.914 062 5	0.965 290 9	0.838 068 9	0.999 508 4	0.889 065 8	0.007 105 2	0.000 552 4	0.847 826 1
randomoversampling_10_90	0.790 540 5	0.921 259 8	0.967 443 9	0.830 201 2	0.999 520 1	0.889 071 3	0.007 161 7	0.000 572 8	0.850 909 1
randomoversampling_20_80	0.804 054 1	0.888 059 7	0.967 999 7	0.834 086 4	0.999 485 0	0.896 611 6	0.007 016 8	0.000 566 6	0.843 971 6
randomundersampling_05_95	0.804 054 1	0.838 028 2	0.971 742 6	0.777 920 5	0.999 391 4	0.896 569 9	0.023 200 9	0.002 721 0	0.820 689 7
randomundersampling_10_90	0.804 054 1	0.782 894 7	0.979 025 1	0.746 662 4	0.999 274 1	0.896 516 6	0.031 607 4	0.003 724 6	0.793 333 3
randomundersampling_20_80	0.770 270 3	0.870 229 0	0.982 100 7	0.728 258 1	0.999 403 1	0.877 563 3	0.048 284 4	0.007 375 9	0.817 204 3
smote_05_95	0.817 567 6	0.876 811 6	0.964 285 1	0.842 948 5	0.999 485 0	0.904 104 3	0.008 192 0	0.000 681 4	0.846 153 8
smote_10_90	0.790 540 5	0.886 363 6	0.977 747 1	0.841 737 4	0.999 461 6	0.889 045 3	0.009 272 3	0.001 046 4	0.835 714 3
smote_20_80	0.804 054 1	0.888 059 7	0.972 445 4	0.842 140 6	0.999 485 0	0.896 611 6	0.008 763 0	0.000 868 0	0.843 971 6
smoteenn_05_95	0.804 054 1	0.901 515 2	0.943 851 1	0.813 384 4	0.999 508 4	0.896 622 4	0.009 114 8	0.000 495 4	0.850 000 0
smoteenn_10_90	0.804 054 1	0.888 059 7	0.961 971 8	0.808 139 0	0.999 485 0	0.896 611 6	0.008 165 5	0.000 576 9	0.843 971 6
smoteenn_20_80	0.797 297 3	0.900 763 4	0.964 547 3	0.834 566 3	0.999 496 7	0.892 847 3	0.009 134 1	0.000 926 3	0.845 878 1
svmsmote_05_95	0.790 540 5	0.921 259 8	0.979 500 3	0.834 966 5	0.999 520 1	0.889 071 3	0.003 435 2	0.000 487 0	0.850 909 1
svmsmote_10_90	0.797 297 3	0.893 939 4	0.984 783 6	0.821 285 3	0.999 485 0	0.892 841 9	0.004 284 5	0.000 577 3	0.842 857 1
svmsmote_20_80	0.777 027 0	0.920 000 0	0.980 100 6	0.822 025 3	0.999 496 7	0.881 439 5	0.004 011 1	0.000 562 3	0.842 490 8

Table 9: Tableau comparatif des configurations de NeuralNet pour la détection de fraude.

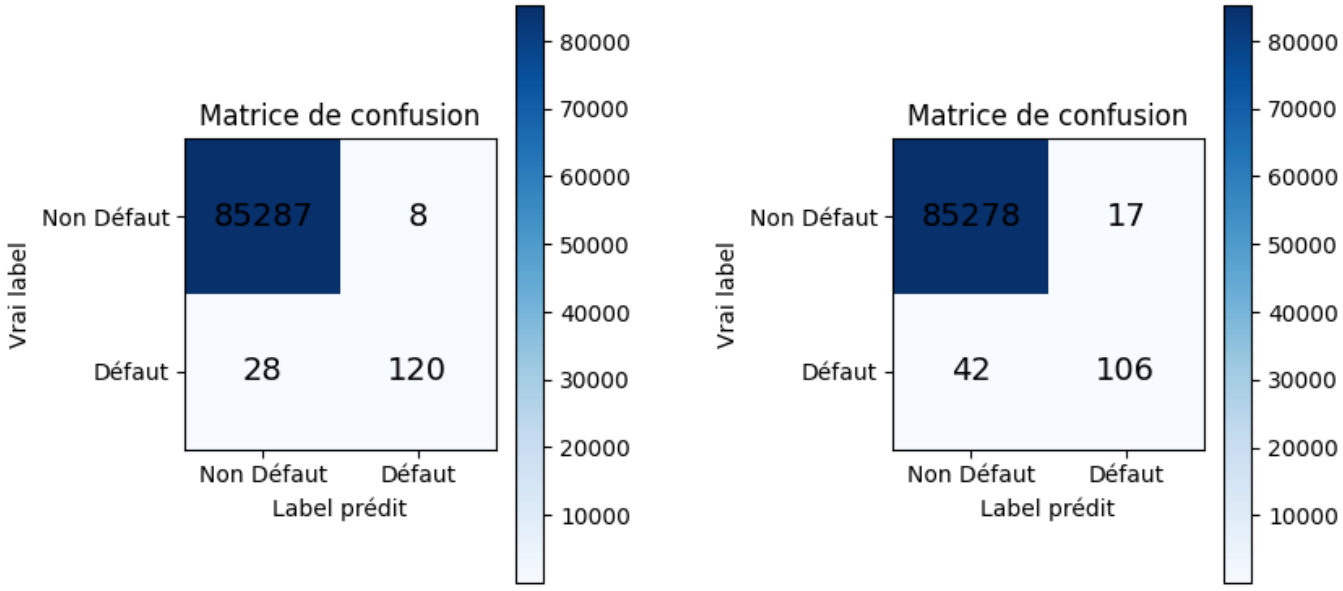
Configuration	Recall	Precision	AUC ROC	AUC PR	Accuracy	G Mean	Cross Entropy	Brier Score	F1 Score
Initiale	0.750 000 0	0.932 773 1	0.985 042 9	0.832 121 7	0.999 473 3	0.865 984 8	0.002 866 4	0.000 490 6	0.831 460 7
adasyn_05_95	0.756 756 8	0.903 225 8	0.970 554 2	0.814 649 3	0.999 438 2	0.869 855 6	0.004 923 6	0.000 622 3	0.823 529 4
adasyn_10_90	0.722 973 0	0.877 049 2	0.951 199 4	0.785 199 8	0.999 344 7	0.850 202 7	0.010 062 3	0.000 692 0	0.792 592 6
adasyn_20_80	0.729 729 7	0.878 048 8	0.960 761 6	0.762 603 1	0.999 356 2	0.854 166 6	0.013 477 2	0.000 674 0	0.797 047 9
borderlinesmote_05_95	0.763 513 5	0.957 627 1	0.965 522 2	0.826 383 1	0.999 531 8	0.873 766 6	0.003 637 9	0.000 528 3	0.849 624 1
borderlinesmote_10_90	0.756 756 8	0.933 333 3	0.978 097 9	0.812 085 3	0.999 485 0	0.869 877 2	0.003 413 0	0.000 545 7	0.835 820 9
borderlinesmote_20_80	0.736 486 5	0.900 826 4	0.958 468 6	0.802 415 3	0.999 403 1	0.858 127 7	0.004 453 3	0.000 598 3	0.810 408 9
cnn	0.729 729 7	0.843 750 0	0.975 430 7	0.757 342 7	0.999 298 2	0.854 142 3	0.035 538 7	0.007 150 5	0.782 608 7
nearmiss_05_95	0.770 270 3	0.844 444 4	0.965 524 0	0.717 713 4	0.999 356 2	0.877 542 0	0.133 764 7	0.034 902 9	0.805 654 5
nearmiss_10_90	0.202 702 7	0.162 162 2	0.942 583 0	0.084 158 1	0.996 804 7	0.449 816 5	0.487 493 6	0.061 484 2	0.180 180 2
nearmiss_20_80	0.668 918 9	0.026 281 1	0.930 256 8	0.017 784 4	0.956 496 6	0.800 096 4	0.435 362 3	0.091 054 0	0.050 574 7
randomoversampling_05_95	0.702 702 7	0.936 936 9	0.954 383 4	0.767 671 2	0.999 403 1	0.838 239 1	0.006 345 7	0.001 189 3	0.803 088 8
randomoversampling_10_90	0.695 945 9	0.944 954 1	0.957 872 9	0.785 724 7	0.999 403 1	0.834 204 2	0.006 842 7	0.000 697 7	0.801 556 4
randomoversampling_20_80	0.736 486 5	0.947 826 1	0.963 654 0	0.793 999 7	0.999 473 3	0.858 158 4	0.008 486 6	0.000 667 7	0.828 897 3
randomundersampling_05_95	0.790 540 5	0.841 726 6	0.975 427 6	0.748 894 0	0.999 380 4	0.889 008 8	0.013 526 7	0.001 674 6	0.815 331 0
randomundersampling_10_90	0.777 027 0	0.851 851 9	0.979 085 3	0.782 033 8	0.999 380 4	0.881 388 4	0.019 158 7	0.003 528 6	0.812 720 8
randomundersampling_20_80	0.743 243 2	0.769 230 8	0.975 693 6	0.719 732 9	0.999 169 0	0.861 949 2	0.040 378 7	0.008 394 7	0.756 013 7
smote_05_95	0.783 783 8	0.811 188 8	0.950 166 9	0.789 752 3	0.999 309 5	0.885 176 0	0.007 300 2	0.000 707 7	0.797 251 4
smote_10_90	0.722 973 0	0.930 434 8	0.950 319 9	0.796 112 3	0.999 426 5	0.850 238 4	0.009 179 8	0.000 614 5	0.813 688 3
smote_20_80	0.743 243 2	0.852 713 2	0.948 072 7	0.771 017 1	0.999 333 0	0.862 020 4	0.007 611 3	0.000 657 2	0.794 223 8
smoteenn_05_95	0.750 000 0	0.917 355 4	0.940 517 5	0.784 487 4	0.999 449 9	0.865 974 8	0.006 633 0	0.000 570 3	0.825 278 8
smoteenn_10_90	0.770 270 3	0.904 761 9	0.967 111 1	0.800 463 0	0.999 461 6	0.877 589 3	0.005 682 6	0.000 673 2	0.832 116 8
smoteenn_20_80	0.750 000 0	0.932 773 1	0.980 598 6	0.823 338 3	0.999 473 3	0.865 984 8	0.004 402 4	0.000 701 7	0.831 460 7
svmsmote_05_95	0.804 054 1	0.782 894 7	0.950 222 3	0.803 445 1	0.999 274 1	0.896 516 6	0.005 819 8	0.000 691 3	0.793 333 3
svmsmote_10_90	0.770 270 3	0.870 229 0	0.956 961 2	0.793 802 0	0.999 403 1	0.877 563 3	0.003 985 9	0.000 623 3	0.817 204 3
svmsmote_20_80	0.770 270 3	0.863 636 4	0.954 807 8	0.788 649 8	0.999 391 4	0.877 558 0	0.004 214 9	0.000 657 4	0.814 285 7

Table 10: Tableau comparatif des configurations de AdaBoost pour la détection de fraude.

Configuration	Recall	Precision	AUC ROC	AUC PR	Accuracy	G Mean	Cross Entropy	Brier Score	F1 Score
Initiale	0.797 297 3	0.797 297 3	0.970 957 8	0.713 417 4	0.999 298 2	0.892 758 0	0.169 694 9	0.028 006 4	0.797 297 3
adasyn_05_95	0.783 783 8	0.800 000 0	0.974 871 1	0.725 834 9	0.999 285 8	0.885 165 5	0.201 734 8	0.038 514 3	0.791 808 9
adasyn_10_90	0.716 216 2	0.868 852 5	0.975 300 6	0.730 204 1	0.999 321 2	0.846 215 7	0.200 284 9	0.037 751 9	0.785 185 2
adasyn_20_80	0.770 270 3	0.832 116 8	0.974 718 3	0.727 944 8	0.999 333 0	0.877 532 4	0.208 438 4	0.040 557 6	0.800 000 0
borderlinesmote_05_95	0.756 756 8	0.848 484 8	0.960 423 2	0.711 280 3	0.999 344 7	0.869 816 3	0.158 169 5	0.023 769 7	0.800 000 0
borderlinesmote_10_90	0.790 540 5	0.769 736 8	0.962 607 2	0.712 906 2	0.999 227 5	0.888 941 3	0.159 749 1	0.023 828 7	0.780 000 0
borderlinesmote_20_80	0.763 513 5	0.801 418 4	0.954 524 3	0.711 094 3	0.999 262 5	0.873 648 6	0.200 349 7	0.035 493 2	0.782 006 9
cnn	0.770 270 3	0.863 636 4	0.970 382 7	0.724 248 4	0.999 391 4	0.877 558 3	0.183 231 0	0.032 430 4	0.814 285 7
nearmiss_05_95	0.770 270 3	0.832 116 8	0.969 368 3	0.711 122 0	0.999 333 0	0.877 532 4	0.171 043 4	0.028 419 3	0.800 000 0
nearmiss_10_90	0.695 945 9	0.851 239 7	0.971 405 6	0.689 691 2	0.999 262 5	0.834 145 6	0.184 963 4	0.032 444 6	0.765 799 3
nearmiss_20_80	0.716 216 2	0.854 838 7	0.970 581 4	0.713 802 7	0.999 298 2	0.846 206 4	0.181 015 9	0.031 496 3	0.779 411 8
randomoversampling_05_95	0.797 297 3	0.797 297 3	0.968 969 4	0.717 834 8	0.999 298 2	0.892 758 0	0.173 047 2	0.028 900 4	0.797 297 3
randomoversampling_10_90	0.777 027 0	0.798 611 1	0.972 747 2	0.722 786 1	0.999 274 1	0.881 342 4	0.195 318 9	0.035 542 9	0.787 671 2
randomoversampling_20_80	0.763 513 5	0.837 037 0	0.973 345 7	0.720 415 7	0.999 333 0	0.873 679 9	0.206 632 0	0.039 005 3	0.798 587 6
randomundersampling_05_95	0.668 918 9	0.860 869 6	0.972 554 7	0.695 902 6	0.999 239 0	0.817 798 5	0.174 557 3	0.029 378 4	0.752 851 7
randomundersampling_10_90	0.777 027 0	0.761 589 4	0.972 898 8	0.706 618 6	0.999 192 4	0.881 305 2	0.201 751 5	0.037 683 4	0.769 230 8
randomundersampling_20_80	0.662 162 2	0.765 625 0	0.966 012 3	0.591 387 4	0.999 064 2	0.813 589 6	0.239 708 9	0.051 118 2	0.710 144 9
smote_05_95	0.777 027 0	0.839 416 1	0.974 141 4	0.731 806 3	0.999 356 2	0.881 378 3	0.192 907 6	0.035 138 5	0.807 017 5
smote_10_90	0.756 756 8	0.794 326 2	0.975 079 0	0.722 830 6	0.999 239 0	0.869 770 1	0.191 329 5	0.035 055 0	0.775 086 5
smote_20_80	0.783 783 8	0.828 571 4	0.974 673 4	0.730 657 4	0.999 344 7	0.885 190 6	0.209 672 3	0.040 636 6	0.805 555 6
smoteenn_05_95	0.763 513 5	0.856 060 6	0.967 041 0	0.724 167 5	0.999 367 9	0.873 695 0	0.172 575 2	0.027 908 2	0.807 142 9
smoteenn_10_90	0.804 054 1	0.793 333 3	0.971 820 8	0.729 426 6	0.999 298 2	0.896 528 3	0.175 857 7	0.029 363 6	0.798 657 7
smoteenn_20_80	0.797 297 3	0.830 985 9	0.974 871 1	0.732 688 2	0.999 367 9	0.892 789 2	0.201 583 2	0.037 342 4	0.813 793 1
svmsmote_05_95	0.750 000 0	0.860 465 1	0.962 774 6	0.703 699 4	0.999 356 2	0.865 933 5	0.157 102 4	0.023 165 9	0.801 444 0
svmsmote_10_90	0.756 756 8	0.848 484 8	0.964 394 2	0.719 500 3	0.999 344 7	0.869 816 3	0.155 978 3	0.022 633 0	0.800 000 0
svmsmote_20_80	0.763 513 5	0.849 624 1	0.969 939 8	0.709 603 8	0.999 356 2	0.873 689 9	0.174 327 9	0.027 338 2	0.804 270 4

Table 11: Tableau comparatif des configurations de XGBoost pour la détection de fraude.

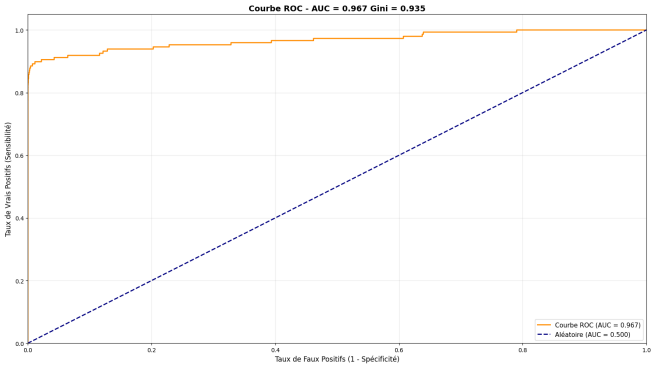
Configuration	Recall	Precision	AUC ROC	AUC PR	Accuracy	G Mean	Cross Entropy	Brier Score	F1 Score
Initiale	0.790 540 5	0.921 259 8	0.977 409 7	0.812 555 5	0.999 520 1	0.889 071 3	0.007 439 5	0.001 169 6	0.850 909 1
adasyn_05_95	0.817 567 6	0.883 211 7	0.972 708 9	0.845 816 5	0.999 496 7	0.904 109 5	0.003 712 4	0.000 553 4	0.849 122 8
adasyn_10_90	0.804 054 1	0.915 384 6	0.968 612 0	0.848 329 0	0.999 531 8	0.896 632 7	0.003 871 0	0.000 515 4	0.856 115 1
adasyn_20_80	0.817 567 6	0.902 985 1	0.969 302 1	0.850 165 5	0.999 531 8	0.904 125 8	0.004 121 0	0.000 506 6	0.858 156 0
borderlinesmote_05_95	0.797 297 3	0.936 507 9	0.978 656 2	0.845 071 9	0.999 555 2	0.892 873 1	0.003 465 1	0.000 442 4	0.861 313 9
borderlinesmote_10_90	0.804 054 1	0.908 396 9	0.971 012 4	0.839 568 8	0.999 520 1	0.896 627 7	0.003 383 4	0.000 486 7	0.853 046 5
borderlinesmote_20_80	0.804 054 1	0.908 396 9	0.971 707 6	0.831 405 4	0.999 520 1	0.896 627 7	0.003 439 5	0.000 458 4	0.853 046 5
cnn	0.804 054 1	0.826 388 9	0.982 078 2	0.776 523 9	0.999 367 9	0.896 559 4	0.019 994 4	0.004 822 0	0.815 068 5
nearmiss_05_95	0.621 621 6	0.828 829 0	0.969 954 7	0.671 891 6	0.999 122 1	0.788 342 3	0.240 069 5	0.064 981 7	0.710 424 7
nearmiss_10_90	0.574 324 3	0.876 288 7	0.968 520 4	0.667 320 8	0.999 122 1	0.757 788 9	0.309 352 3	0.079 389 2	0.693 877 6
nearmiss_20_80	0.675 675 7	0.793 650 8	0.967 521 4	0.681 085 8	0.999 133 8	0.821 870 4	0.387 069 3	0.103 933 0	0.729 927 0
randomoversampling_05_95	0.804 054 1	0.922 480 6	0.976 620 2	0.839 705 1	0.999 543 5	0.896 638 4	0.003 921 7	0.000 453 3	0.859 206 5
randomoversampling_10_90	0.797 297 3	0.929 133 9	0.974 958 2	0.838 215 9	0.999 543 5	0.892 868 0	0.003 711 0	0.000 456 6	0.858 181 8
randomoversampling_20_80	0.810 810 8	0.909 090 9	0.975 487 8	0.836 619 7	0.999 531 8	0.900 386 7	0.003 665 3	0.000 455 4	0.857 142 9
randomundersampling_05_95	0.763 513 5	0.896 825 4	0.976 050 6	0.811 460 4	0.999 438 2	0.873 726 1	0.008 091 0	0.001 590 5	0.824 817 5
randomundersampling_10_90	0.783 783 8	0.828 571 4	0.981 262 4	0.814 717 4	0.999 344 7	0.885 190 6	0.013 724 9	0.003 227 6	0.805 555 6
randomundersampling_20_80	0.729 729 7	0.900 000 0	0.980 040 3	0.745 186 9	0.999 391 4	0.854 182 2	0.033 242 8	0.008 511 0	0.805 970 1
smote_05_95	0.824 324 3	0.903 703 7	0.969 877 1	0.845 036 5	0.999 543 5	0.907 853 9	0.003 951 6	0.000 504 1	0.862 190 8
smote_10_90	0.810 810 8	0.937 500 0	0.967 424 4	0.852 445 4	0.999 578 6	0.900 407 8	0.003 812 2	0.000 498 8	0.869 565 2
smote_20_80	0.790 540 5	0.943 548 4	0.968 778 3	0.851 519 6	0.999 555 2	0.889 087 2	0.003 698 4	0.000 574 2	0.860 294 1
smoteenn_05_95	0.810 810 8	0.909 090 9	0.961 564 0	0.842 396 5	0.999 531 8	0.900 386 7	0.003 998 2	0.000 474 6	0.857 142 9
smoteenn_10_90	0.790 540 5	0.921 259 8	0.957 542 0	0.837 851 1	0.999 520 1	0.889 071 3	0.004 033 1	0.000 556 4	0.850 909 1
smoteenn_20_80	0.777 027 0	0.934 959 3	0.975 450 6	0.845 387 5	0.999 520 1	0.881 449 5	0.004 035 6	0.000 550 4	0.848 708 5
svmsmote_05_95	0.804 054 1	0.922 480 6	0.979 863 5	0.833 831 3	0.999 543 5	0.896 638 4	0.003 329 5	0.000 456 1	0.859 206 5
svmsmote_10_90	0.804 054 1	0.937 007 9	0.966 899 0	0.842 950 0	0.999 566 9	0.896 648 6	0.003 255 8	0.000 438 2	0.865 455 0
svmsmote_20_80	0.797 297 3	0.936 507 9	0.976 248 8	0.835 991 6	0.999 555 2	0.892 873 1	0.003 516 5	0.000 437 1	0.861 313 9



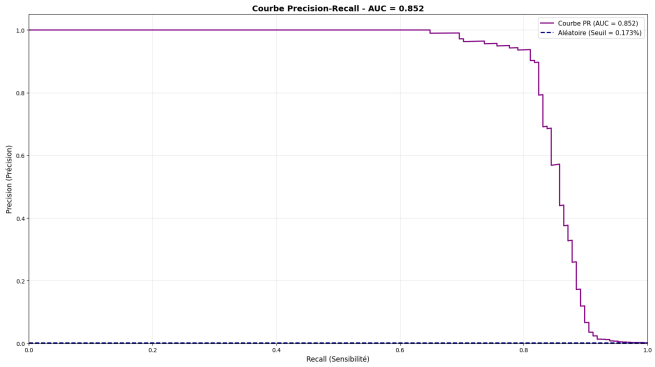
(a) Meilleur modèle

(b) Approche Cost-Sensitive

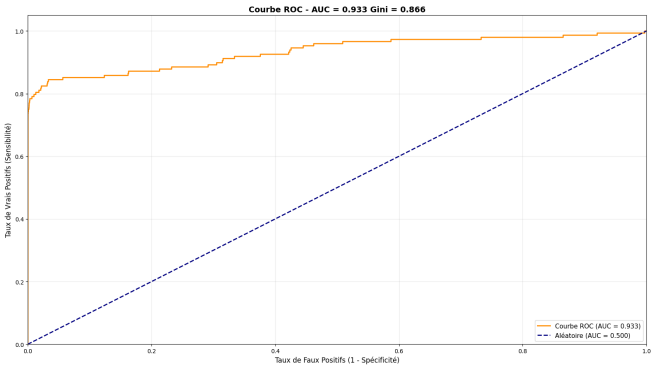
Figure 6: Matrices de Confusion



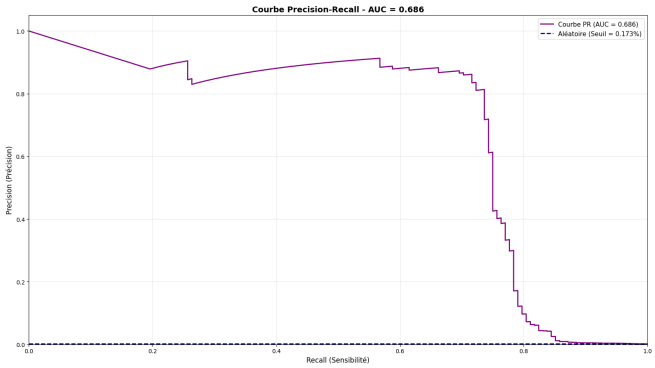
(a) Courbe ROC - Meilleur modèle



(b) Courbe Precision-Recall - Meilleur modèle

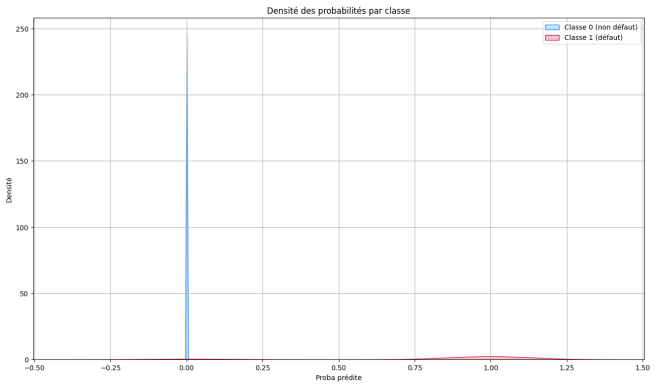


(c) Courbe ROC - Cost-Sensitive

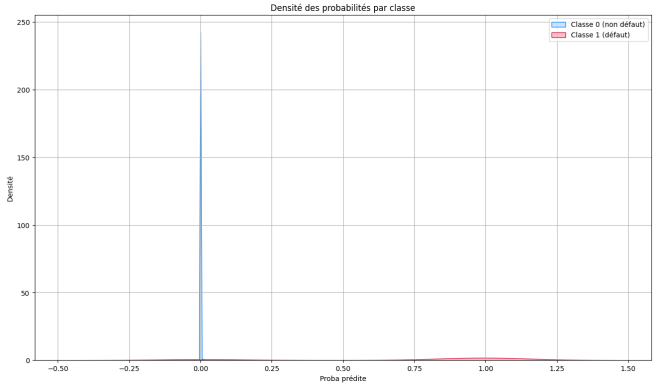


(d) Courbe Precision-Recall - Cost-Sensitive

Figure 7: Comparaison des courbes ROC et Precision-Recall.



(a) Meilleur modèle



(b) Approche Cost-Sensitive

Figure 8: Densité des probabilités prédites par classe.

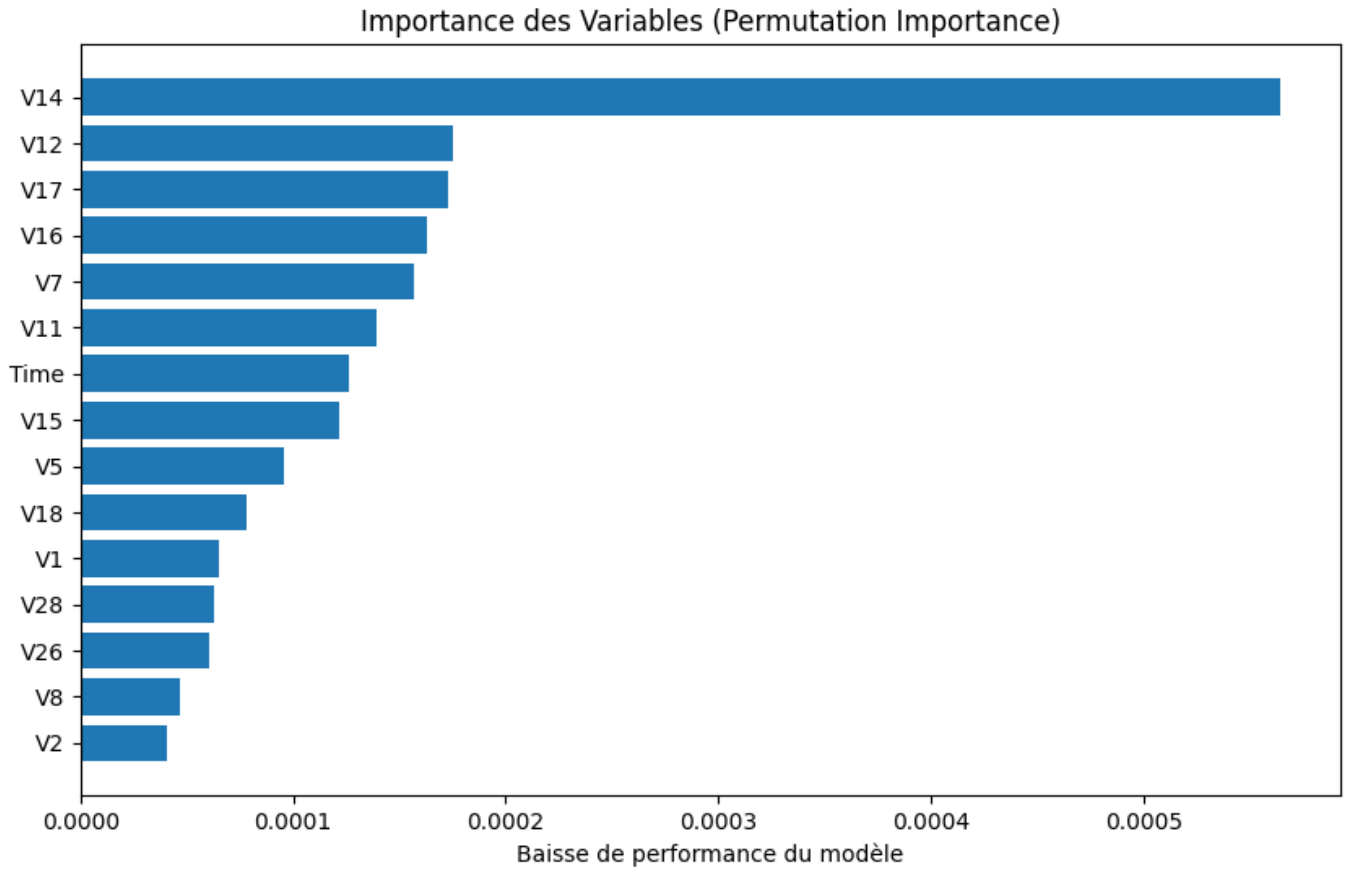


Figure 9: Variables importantes XGBoost SMOTE 10/90